



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
ASSIGNMENT - 06

Problem Statement

Write a Python program to solve the $(n^2 - 1)$ -puzzle problem using the following Informed Search algorithms:

- **A* Search**
- **Greedy Best-First Search**

The program should:

1. Accept n as input ($n = 3$ for 8-puzzle, $n = 4$ for 15-puzzle).
2. Generate **10 random solvable initial states**. (Recommendation: Use the shuffling method from Assignment 05 to ensure solvability).
3. Implement and test **three specific heuristics** for both algorithms:
 - h_1 (**Misplaced Tiles**): The number of tiles not in their goal position.
 - h_2 (**Manhattan Distance**): The sum of the vertical and horizontal distances of tiles from their goal positions.
 - h_3 (**Custom Heuristic**): A custom heuristic of your design. It **must** be admissible.
4. Set a **strict 1-minute timeout** per algorithm/heuristic combination per state.
5. Measure **Execution Time** and **Nodes Expanded** (visited).

Experimental Requirements

1. Constraints on Environment

To ensure the focus remains on algorithm logic, you **must not** use an IDE.

- Use a basic text editor (e.g., Notepad, Gedit, Vim) without code autocompletion or AI assistance.
- Execution and timing must be handled via the command line.

2. Data Collection & Plotting

Generate the following plots comparing the algorithms across different values of n (e.g., $n = 3$ and $n = 4$):

- **Average Execution Time:** Compare A* vs. Greedy Best-First for each of the three heuristics.
- **Nodes Expanded:** A graph showing the average number of nodes added to the closed list (visited set).

Note: If an algorithm times out, exclude that run from the average calculations.

Key Technical Observations

Students should observe the following during their experiments:

- **Optimality vs. Speed:** A* with an admissible heuristic guarantees an optimal solution. Does Greedy Best-First Search find the solution faster? If so, is the path length found by Greedy Best-First optimal?
- **Heuristic dominance:** Compare h_1 (Misplaced) and h_2 (Manhattan). Which heuristic results in fewer nodes being expanded? Relate this to the concept of one heuristic dominating another.
- **Custom Heuristic:** Explain your choice of h_3 . How does its performance compare to the standard heuristics?
- **Scalability:** How does the memory usage (nodes expanded) change for $n = 4$ compared to the uninformed searches used in Assignment 04?

Submission Requirements

- **Source Code:** A single Python file implementing the puzzle, heuristics, and both search algorithms.
- **Report:** A PDF including:
 1. The required plots (Time and Nodes Expanded).